

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ  
Національний університет «Запорізька політехніка»

Факультет комп'ютерних наук і технологій  
(повне найменування факультету)

Кафедра програмних засобів  
(повне найменування кафедри)

## Пояснювальна записка

до дипломного проекту (роботи)

бакалавр

(ступінь вищої освіти)

на тему Розроблення програмного забезпечення інтернет-магазину

вантажівок

Trucks Online Store

Виконав(ла): студент(ка) 4 курсу, групи КНТ-122

Спеціальності 121 Інженерія програмного

(код і найменування спеціальності)

забезпечення

Освітня програма (спеціалізація)

Інженерія програмного забезпечення

ОНИЩЕНКО О.А.

(ПРИЗВИЩЕ та ініціали)

Керівник СТЕПАНЕНКО О.О.

(ПРИЗВИЩЕ та ініціали)

Рецензент СИДОРЕНКО С.С.

(ПРИЗВИЩЕ та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ  
Національний університет «Запорізька політехніка»  
(повне найменування закладу вищої освіти)

Факультет КНТ  
Кафедра програмних засобів  
Ступінь вищої освіти бакалавр  
Спеціальність 121 Інженерія програмного забезпечення  
(код і найменування)  
Освітня програма (спеціалізація) Інженерія програмного забезпечення  
(назва освітньої програми (спеціалізації))

**ЗАТВЕРДЖУЮ**

Завідувач кафедри ПЗ, д.т.н, проф.  
Сергій СУББОТІН  
“ ” 2026 року

**З А В Д А Н Н Я**

**НА ДИПЛОМНИЙ ПРОЄКТ (РОБОТУ) СТУДЕНТА(КИ)**

ОНИЩЕНКО Олега Антоновича

(ПРІЗВИЩЕ, ім'я, по батькові)

1. Тема проєкту (роботи) Розроблення програмного забезпечення інтернет-магазину вантажівок. Trucks Online Store  
керівник проєкту (роботи) к.т.н., доцент, СТЕПАНЕНКО Олександр Олексійович,

(науковий ступінь, вчене звання, ПРІЗВИЩЕ, ім'я, по батькові)

затверджені наказом закладу вищої освіти від “ 14 ” січня 2026 року № 1

2. Строк подання студентом проєкту (роботи) 10 червня 2026 року

3. Вихідні дані до проєкту (роботи) рекомендована література, технічне завдання

4. Зміст розрахунково-пояснювальної записки (перелік питань, які потрібно розробити) 1. Аналіз проблеми та постановка завдань дослідження. 2. Матеріали і методи. 3. Опис програми. 4. Експлуатація, тестування та експериментальне дослідження програми.

5. Перелік графічного матеріалу (з з точним зазначенням обов'язкових креслень, кількість слайдів, плакатів)

Слайди презентації

## 6. Консультанти розділів проєкту (роботи)

| Розділ              | ПРИЗВИЩЕ, ініціали та посада консультанта | Підпис, дата   |                           |
|---------------------|-------------------------------------------|----------------|---------------------------|
|                     |                                           | завдання видав | прийняв виконане завдання |
| 1-4 Основна частина | СТЕПАНЕНКО О.О., доцент                   |                |                           |
| Нормоконтроль       | ДМИТРЕНКО Д.Д., ст. викладач              |                |                           |

7. Дата видачі завдання « 14 » січня 2024 року.

## КАЛЕНДАРНИЙ ПЛАН

| № з/п | Назва етапів дипломного проєкту (роботи)                             | Строк виконання етапів проєкту ( роботи ) | Примітка     |
|-------|----------------------------------------------------------------------|-------------------------------------------|--------------|
| 1     | Постановка завдання роботи.                                          | 1 тиждень                                 | Завдання, ТЗ |
| 2     | Аналіз предметної області.                                           | 1 тиждень                                 | Розділ 1     |
| 3     | Вибір мови програмування та інших технологій розробки.               | 2 тиждень                                 | Розділ 2     |
| 4     | Розробка архітектури програми.                                       | 2 тиждень                                 | Розділ 3     |
| 5     | Розробка програми.                                                   | 3-4 тижні                                 | Розділ 3, 4  |
| 6     | Тестування та експериментальне дослідження програмного забезпечення. | 5 тиждень                                 | Розділ 4     |
| 7     | Оформлення пояснювальної записки та документів до неї.               | 6 тиждень                                 | Додатки      |
| 8     | Нормоконтроль та рецензування.                                       | 7 тиждень                                 |              |
| 9     | Захист роботи.                                                       | 8 тиждень                                 |              |

Студент(ка)

\_\_\_\_\_  
( підпис )      Олег ОНИЩЕНКО  
(Ім'я ПРИЗВИЩЕ)

Керівник проєкту (роботи)

\_\_\_\_\_  
( підпис )      Олександр СТЕПАНЕНКО  
(Ім'я ПРИЗВИЩЕ)

## РЕФЕРАТ

Пояснювальна записка до дипломної кваліфікаційної роботи бакалавра містить: 55 сторінок, 8 таблиць, 20 рисунків, 3 додатки, 6 джерел.

ІНТЕРНЕТ МАГАЗИН ВАНТАЖІВОК, МАГАЗИН ТОВАРІВ, ВЕБ-ЗАСТОСУНОК, NICEGUI, PYTHON.

Об'єкт проєктування – процес розробки програмного забезпечення про продаж вантажівок.

Предмет проєктування – методи та програмні засоби купівлі та продажу вантажівок.

Мета роботи – прискорення пошуку вантажівок для купівлі.

Матеріали, методи та технічні засоби – використання відкритих джерел інформації та відомих засобів розробки.

Результатом роботи має бути готова програма, пояснювальна записка, презентація роботи.

Програмний застосунок містить наступні функції:

- купівля вантажівки;
- продаж вантажівки;
- здійснення рейсів.

Застосунок може бути використаний на таких платформах:

- Windows;
- MacOS;
- Android;
- iOS.

## ABSTRACT

The explanatory note to the bachelor's thesis contains: 55 pages, 8 tables, 20 figures, 3 appendices, 6 sources.

INTERNET TRUCK STORE, GOODS STORE, WEB APPLICATION, NICEGUI, PYTHON.

The object of design is modern online truck stores.

The subject of design is software for selling trucks on the Internet.

The goal of the work is to create a software system for ordering trucks.

Materials, methods, and technical means: use of open sources of information and well-known development tools.

The result of the work should be a finished program, explanatory note, and project presentation.

The software application includes the following functions:

- truck purchase;
- truck sale;
- trip execution.

The application can be used on the following platforms:

- Windows;
- MacOS;
- Android;
- iOS.

## ЗМІСТ

|                                                    |    |
|----------------------------------------------------|----|
| 1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ОГЛЯД АНАЛОГІВ..... | 10 |
| 2 ПРОЄКТУВАННЯ СТРУКТУРИ СИСТЕМИ.....              | 17 |
| 3 ПРОГРАМНА РЕАЛІЗАЦІЯ.....                        | 19 |
| 4 РОЗРОБКА ІНТЕРФЕЙСУ.....                         | 21 |
| 5 КЕРІВНИЦТВО КОРИСТУВАЧА.....                     | 27 |
| ВИСНОВКИ.....                                      | 33 |
| ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ.....                      | 34 |
| ДОДАТОК А.....                                     | 35 |
| ДОДАТОК Б.....                                     | 38 |
| ДОДАТОК В.....                                     | 46 |

## ПЕРЕЛІК СКОРОЧЕНЬ ТА УМОВНИХ ПОЗНАК

|         |                                                          |
|---------|----------------------------------------------------------|
| NiceGUI | Бібліотека створення графічних інтерфейсів мовою Python  |
| Python  | Високорівнева мова програмування з динамічною типізацією |

## ВСТУП

Перевезення є важливою галуззю в світі. Люди щодня мають потреби. Їм потрібно їсти, пити та вдягатися (1 Тимофія 6:8). Для цього існують магазини, де люди купують товари. Місцеві магазини потребують доставки товару. Ланцюг доставки приблизно виглядає так:

1. товар виробляється на заводах;
2. з заводів товар перевозиться на склади;
3. зі складів товари розвозяться по магазинах.

Аби працювати на ринку перевезень потрібно або працювати на вантажівці компанії, або придбати власну. Власну вантажівку можна придбати на різних вебсайтах. Кілька відомих брендів вантажівок:

- DAF;
- Volvo;
- Scania.

Кожна з цих компаній має веб-сайт, на якому можна придбати (або подивитися де можна придбати) власну вантажівку.

Програма корисна тим, що об'єднує в собі всі місця пошуку автомобілів і дозволяє користувачеві їх придбати.

## **1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ОГЛЯД АНАЛОГІВ**

### **1.1 Огляд існуючих методів вирішення завдання**

Вантажні перевезення є важливою галуззю у світі. Для вантажних перевезень потрібні автомобілі. Вантажні автомобілі бувають таких типів [1]:

- тент;
- трал;
- бус;
- евакуатор;
- маніпулятор;
- рефрижератор.

Знайдені застосунки є застосунками, які допомагають купити власний вантажний автомобіль. Вони є гарними прикладами зручного функціоналу та світлого графічного інтерфейсу користувача.

### **1.2 Огляд існуючих програмних засобів**

#### ***1.2.1 Веб-застосунок Auto.RIA***

Застосунок Auto.RIA[1] є популярним веб-застосунком для продажу та купівлі автомобілів. Він містить різні фільтри. Деякі з них нижче:

- наявність відео машини;
- регіон;
- чи була у ДТП;
- технічний стан;
- ціна.

Переваги:

- зрозумілий інтерфейс;
- швидка взаємодія;
- багато пропозицій.

Вигляд інтерфейсу веб-застосунку на рисунку нижче:



### 1.2.2 Веб-застосунок Volvo

Веб-сайт Volvo[2] є офіційним сайтом компанії. Корисною функцією є пошук автосалонів. Веб-застосунок має світлий інтерфейс та можливість зібрати власну вантажівку у конфігураторі.

Переваги:

- зрозумілий інтерфейс;
- наявність пошуку автосалонів.

Вигляд застосунку на рисунку нижче:

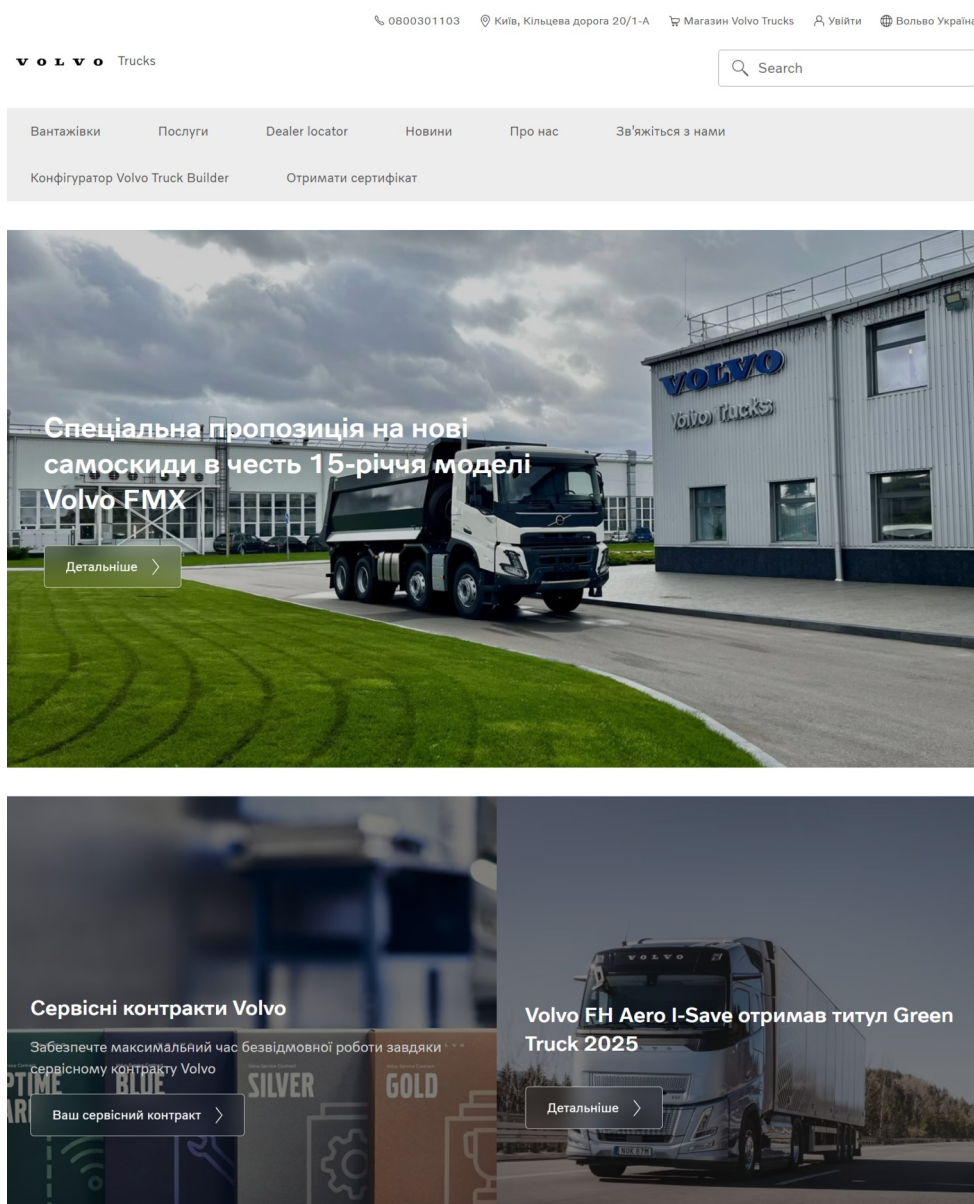


Рисунок 1.2 — Вигляд веб-застосунку

### 1.2.3 Веб-застосунок DAF

Веб-сайт DAF[3] також є офіційним сайтом компанії. Він має гарний інтерфейс та можливість пошуку автосалонів.

Переваги:

- гарний інтерфейс;
- можливість знайти найближчий автосалон.

Вигляд застосунку наведено на рисунку нижче:

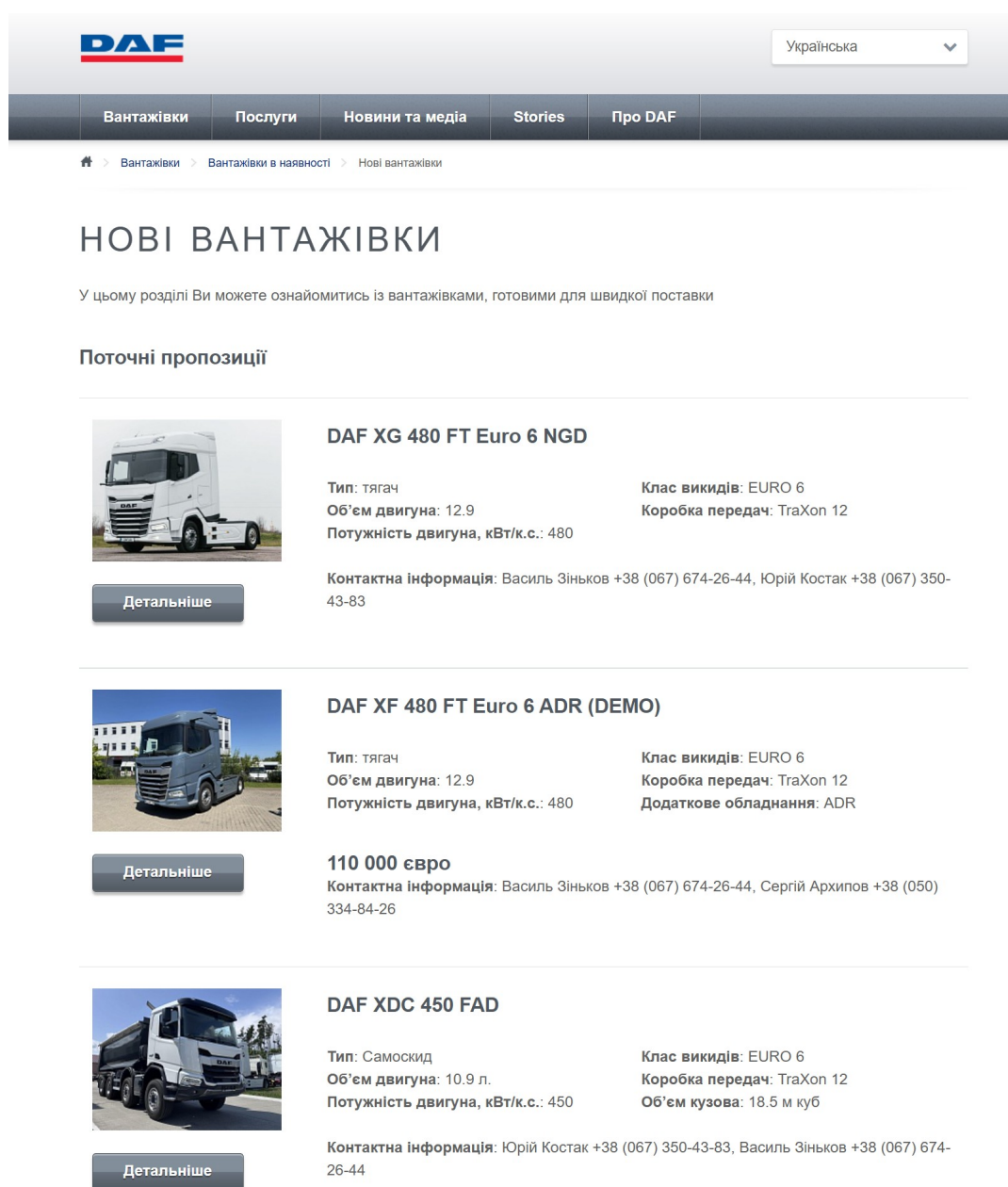


Рисунок 1.3 — Вигляд сторінки продажу

### 1.2.4 Веб-застосунок RST

Веб-застосунок RST[4] є веб-застосунком продажу та купівлі автомобілів. Він має зрозумілий інтерфейс та численні фільтри.

Переваги:

- зрозумілий інтерфейс;
- доступні фільтри;
- можливість авторизації.

Вигляд застосунку наведено нижче на рисунку:

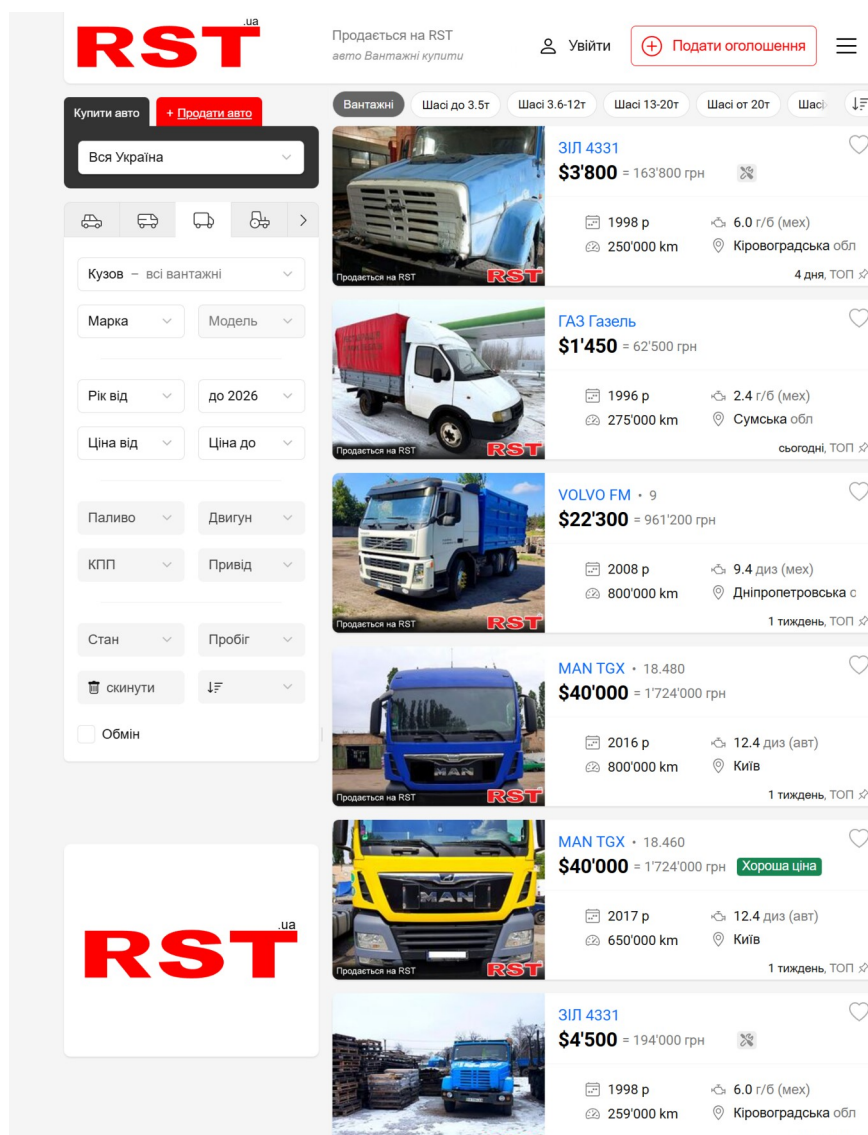


Рисунок 1.4 — Вигляд веб-застосунку

### 1.2.5 Веб-застосунок OLX

Веб-застосунок OLX[5] містить велику кількість фільтрів, гарний користувацький інтерфейс та можливість написати продавцеві у чат.

Переваги:

- велика кількість фільтрів;
- світлий інтерфейс;
- можливість написати продавцеві у чат.

Вигляд застосунку наведено на рисунку нижче:

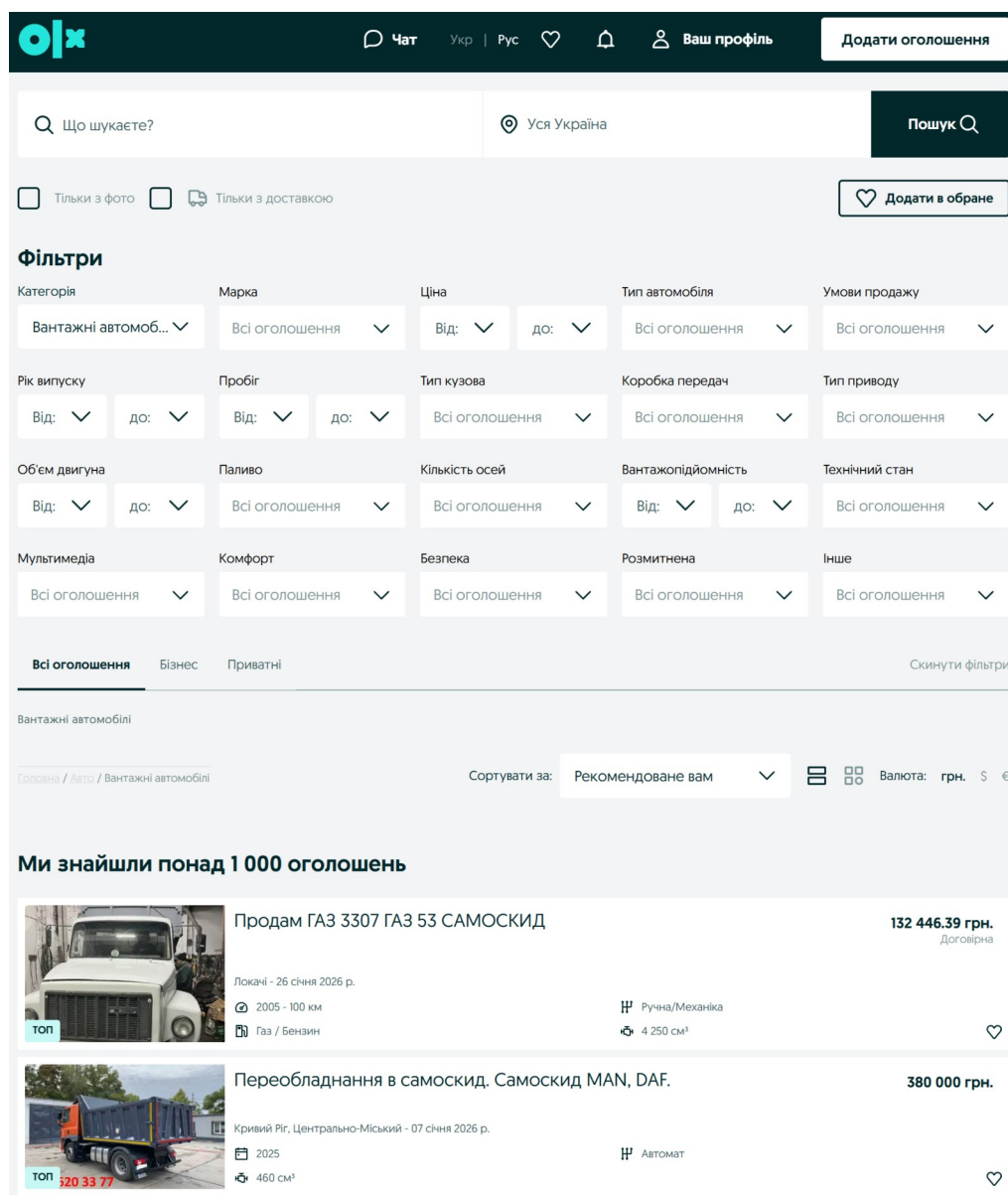


Рисунок 1.5 — Вигляд веб-застосунку

### 1.3 Порівняння застосунків

Порівняння застосунків наведено на таблиці нижче:

Таблиця 1.1 — Порівняльна таблиця застосунків

| Критерій                    | AutoRIA | Volvo | DAF | RST | OLX |
|-----------------------------|---------|-------|-----|-----|-----|
| Інтерфейс                   | +       | +     | +   | -   | -   |
| Можливість знайти автосалон | -       | +     | +   | -   | -   |
| Можливість авторизації      | +       | +     | -   | +   | +   |

### 1.4 Висновки за розділом

Отож, всі розглянуті застосунки містять світлий графічний інтерфейсу користувача та багато корисних функцій для спрощення пошуку вантажівок.

## 2 ПРОЄКТУВАННЯ СТРУКТУРИ СИСТЕМИ

### 2.1 Створення структурної схеми

Схеми бази даних з описами наведено нижче на таблицях:

Таблиця 2.1 — Модель Truck

| Властивість | Тип даних | Опис                                   |
|-------------|-----------|----------------------------------------|
| Name        | String    | Назва вантажівки                       |
| Category    | String    | Категорія: Бус   Вантажівка   Всюдихід |
| Price       | Integer   | Ціна вантажівки                        |
| Picture     | String    | Назва картинки вантажівки              |

Таблиця 2.2 — Модель Loan

| Властивість | Тип даних | Опис                           |
|-------------|-----------|--------------------------------|
| Amount      | Integer   | Розмір кредиту в грошах        |
| Duration    | Integer   | Тривалість кредиту (місяці)    |
| Bank        | String    | Назва банку, який видає кредит |

Таблиця 2.3 — Модель Order

| Властивість | Тип даних          | Опис                                           |
|-------------|--------------------|------------------------------------------------|
| Price       | Integer            | Вартість замовлення                            |
| Location    | String             | Назва населеного пункту, куди замовлення везти |
| Coordinates | Pair<float, float> | Координати населеного пункту                   |

Структурну схему бази даних наведено нижче на рисунку:

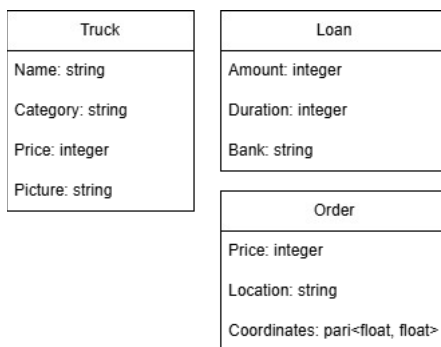


Рисунок 2.1 — Структурна схема

## 2.2 Створення функціональної схеми

Розроблений веб-застосунок має включати такі функції:

- купівля вантажівок;
- взяття кредиту;
- здійснення замовлень.

Схему функцій наведено на рисунку нижче:

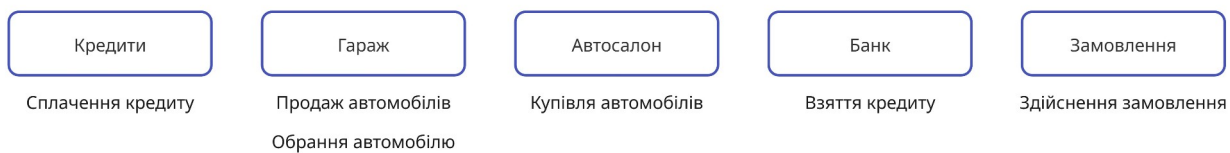


Рисунок 2.2 — Функціональна схема

## 3 ПРОГРАМНА РЕАЛІЗАЦІЯ

### 3.1 Вибір засобів розробки

Для розробки серверної частини обрано мову програмування Python та бібліотеку NiceGUI. Ця бібліотека інтегрує в себе TailwindCSS для стилізації та багато різних віджетів.

#### 3.1.1 Обґрунтування вибору засобів розробки серверної частини

Мову програмування Python обрано через її легкий синтаксис, наявність великої кількості бібліотек та широку підтримку спільноти.

Порівняння мови програмування Python з іншими опціями наведено нижче на таблиці:

Таблиця 3.1 — Порівняння мов програмування

| Критерій            | Python    | Java     | C#       |
|---------------------|-----------|----------|----------|
| Типізація           | Динамічна | Статична | Статична |
| Наявність бібліотек | Висока    | Середня  | Середня  |
| Підтримка спільноти | Висока    | Середня  | Середня  |

Фреймворк для мови Python обрано NiceGUI. Він використовується для створення веб-застосунків різних типів. Основними перевагами фреймворку є його простота застосування та наявність багатьох бібліотек.

Порівняння фреймворку NiceGUI з іншими опціями наведено на таблиці нижче:

Таблиця 3.2 — Порівняння фреймворків для Python

| Критерій           | NiceGUI | Streamlit | Tkinter |
|--------------------|---------|-----------|---------|
| Платформа          | Web     | Web       | Desktop |
| Персоналізація     | Висока  | Низька    | Середня |
| Швидкість розробки | Висока  | Середня   | Середня |

### 3.1.2 Обґрунтування вибору середовища розробки

Середовище розробки VS Code обрано через його популярність, легкість налаштування та підтримку спільноти.

Інші варіанти порівняно на таблиці нижче:

Таблиця 3.5 — Порівняння інтегрованих середовищ розробки

| Критерій              | Visual Studio Code | WebStorm |
|-----------------------|--------------------|----------|
| Простота              | Висока             | Середня  |
| Кількість налаштувань | Середня            | Висока   |
| Підтримка спільноти   | Висока             | Середня  |

## 3.2 Висновки за розділом

Отже, обрані засоби розробки було обґрунтовано та порівняно з їхніми аналогами.

## 4 РОЗРОБКА ІНТЕРФЕЙСУ

### 4.1 Вибір платформи

Як платформу для веб-застосунку обрано веб-браузер через його поширеність та можливість розробки одразу і під мобільні пристрої.

### 4.2 Розробка макету інтерфейсу

Попередній вигляд інтерфейсу наведено на рисунку нижче:



Рисунок 4.1 — Вигляд сторінки автосалону



Рисунок 4.2 — Вигляд сторінки банку

### 4.3 Вигляд розробленого інтерфейсу

Вигляд різних сторінок інтерфейсу веб-застосунку наведено нижче на рисунках:

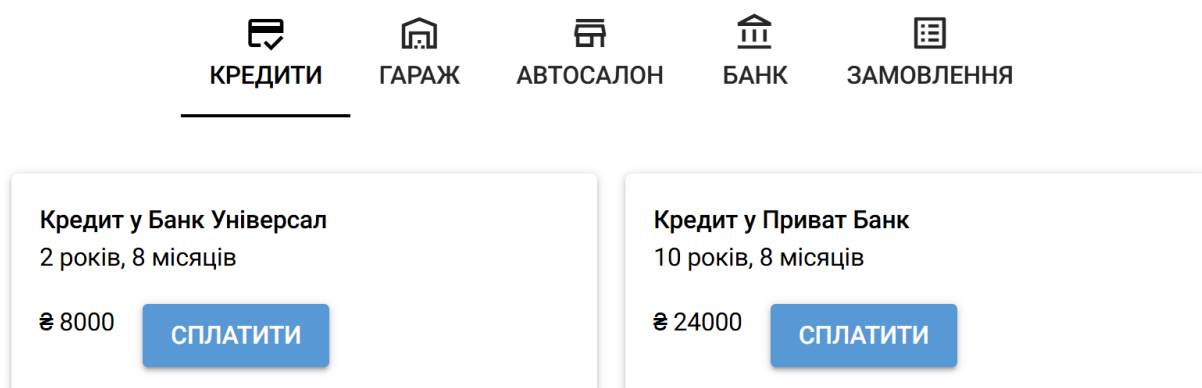


Рисунок 4.3 — Вигляд сторінки кредитів

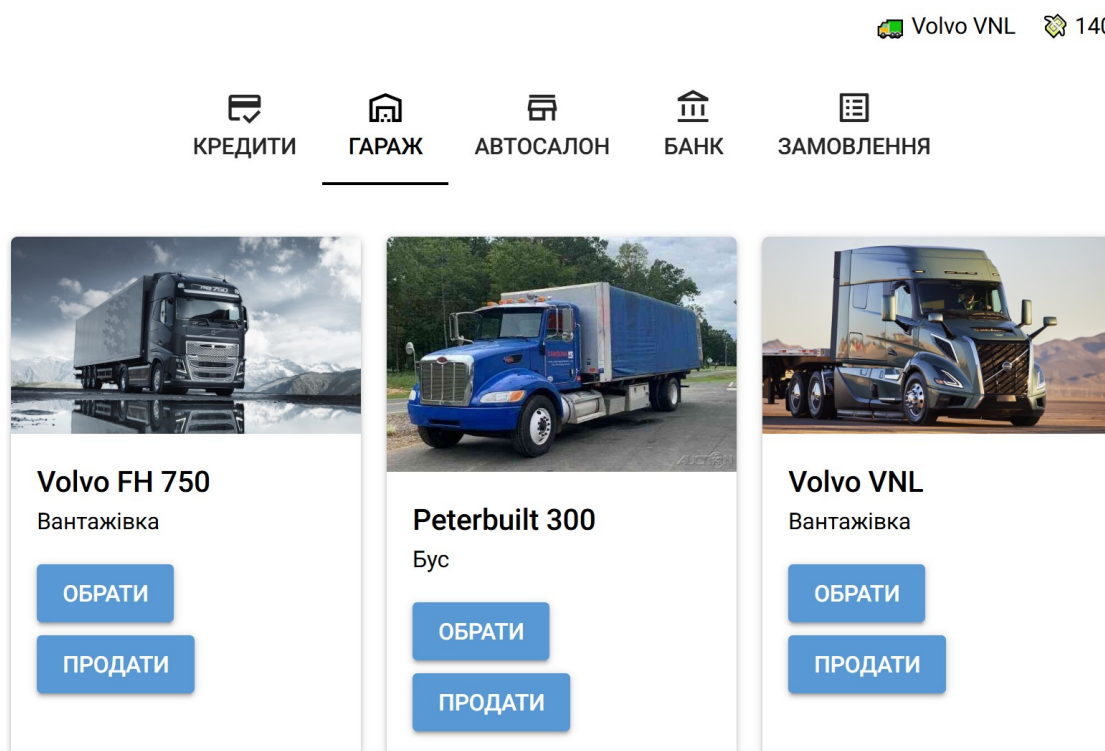


Рисунок 4.4 — Вигляд сторінки гаражу

КРЕДИТИ

ГАРАЖ

АВТОСАЛОН

БАНК

ЗАМОВЛЕННЯ



DAF XF

Бус

₴ 4000

КУПИТИ



DAF XF

Вантажівка

₴ 7000

КУПИТИ



Mercedes Actros 7

Вантажівка

₴ 6000

КУПИТИ



Mercedes Actros 5

Вантажівка

₴ 6000

КУПИТИ



Mercedes Sprinter Offroad

Всюдихід

₴ 4000

КУПИТИ



Renault Master

Бус

₴ 5000

КУПИТИ



Renault Master

Бус

₴ 5000

КУПИТИ



Volvo FH 750

Вантажівка

₴ 6000

КУПИТИ



Volvo VNL

Вантажівка

₴ 7000

КУПИТИ



Scania R500

Вантажівка

₴ 8000

КУПИТИ



Peterbuilt 500

Вантажівка

₴ 8000

КУПИТИ



Peterbuilt 300

Бус

₴ 5000

КУПИТИ

Рисунок 4.5 — Вигляд сторінки автосалону

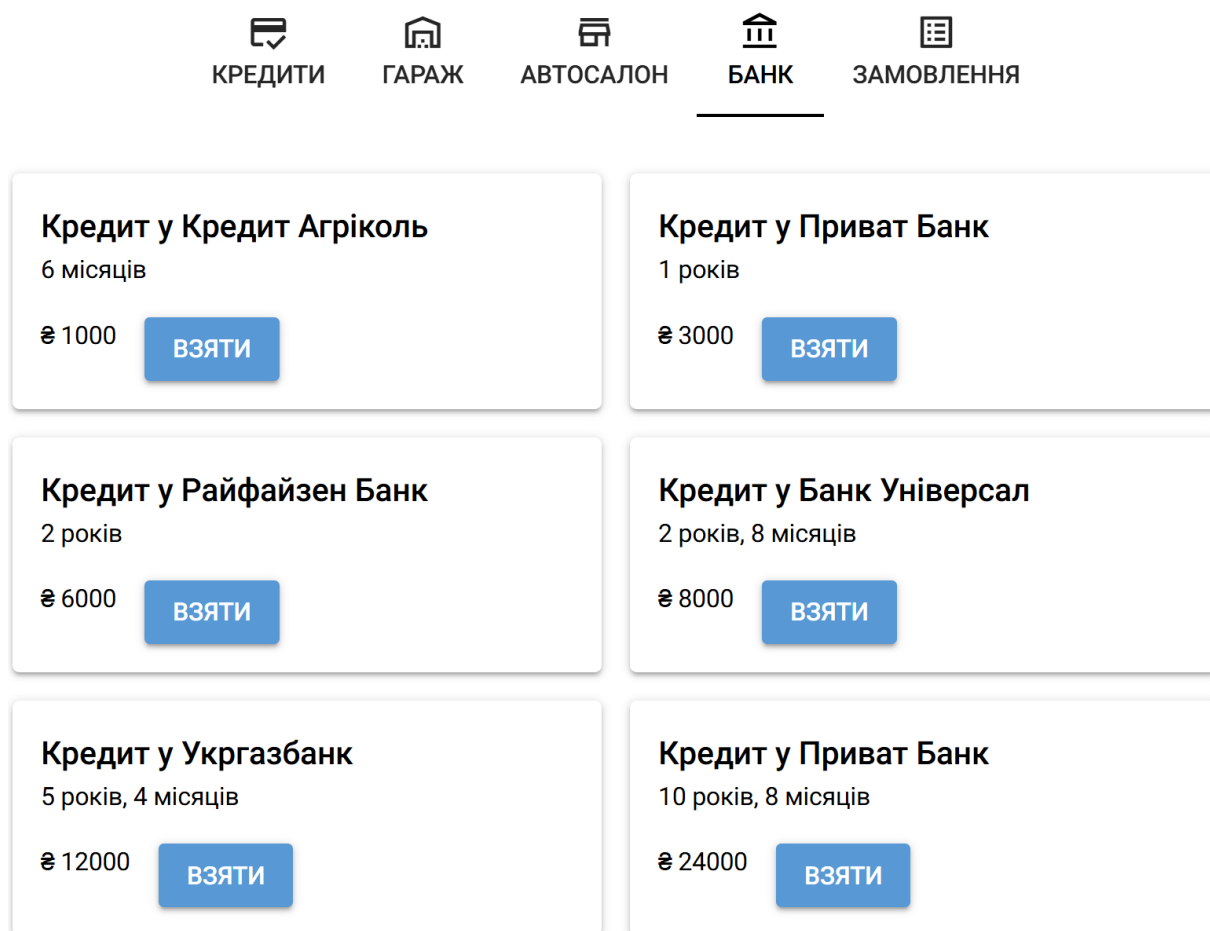


Рисунок 4.6 — Вигляд сторінки банку

**ВИКОНАТИ****ВИКОНАТИ****ВИКОНАТИ****ВИКОНАТИ****ВИКОНАТИ****ВИКОНАТИ****ВИКОНАТИ****ВИКОНАТИ****ВИКОНАТИ****ВИКОНАТИ****ВИКОНАТИ****ВИКОНАТИ**

Рисунок 4.7 — Вигляд сторінки замовлень

#### **4.4 Висновок за розділом**

Отже, графічний інтерфейс користувача було спроектовано та розроблено раніше обраними програмними засобами.

## 5 КЕРІВНИЦТВО КОРИСТУВАЧА

### 5.1 Необхідні умови для виконання

Для виконання програми потрібно:

1. завантажити архів програми собі на систему;
2. розархівувати код програми;
3. завантажити та встановити Python останньої версії.

Аби запустити програму потрібно:

1. з теки програми ввести команду `.venv\Scripts\activate.bat`, далі ввести `python main.py`;
3. перейти до сторінки <http://localhost:8080>.

### 5.2 Звертання до програми

Програма складається з п'яти сторінок. На початку показується сторінка автосалону.

Опис сторінок наведено на таблиці нижче:

Таблиця 5.1 — Опис кожної сторінки

| Сторінка   | Опис                                        |
|------------|---------------------------------------------|
| Кредити    | Взяті кредити, де їх можна сплатити         |
| Гараж      | Куплені вантажівки, де їх можна переглянути |
| Автосалон  | Магазин вантажівок, де їх можна купити      |
| Банк       | Сторінка кредитів, де їх можна взяти        |
| Замовлення | Сторінка замовлень, де їх можна виконати    |

Верхнє меню містить значки грошей та обраної вантажівки.

Значок грошей позначено символом , а значок вантажівки символом .

Значок грошей показує поточний баланс користувача. Значок вантажівки показує обрану вантажівку, на якій здійснюються замовлення.

### 5.2.1 Сторінка кредитів

Сторінка кредитів виглядає наступним чином:

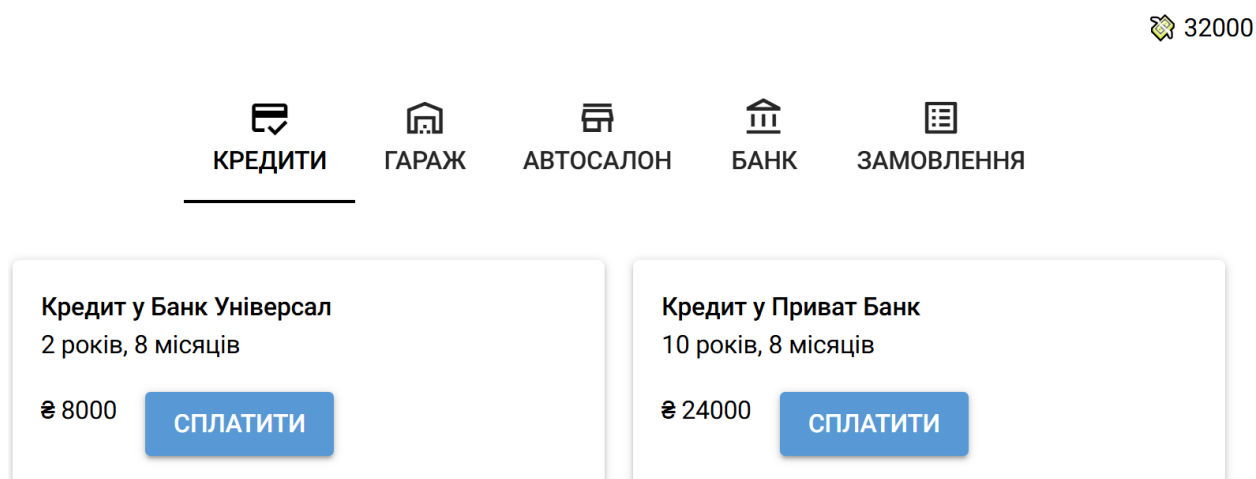


Рисунок 5.1 — Вигляд сторінки кредитів

На цій сторінці відображаються взяті кредити. Їх можна оплатити, натискаючи на кнопку Сплатити під відповідним кредитом.

Коли кредити відсутні, сторінка виводить наступне повідомлення:



Немає взятих кредитів.

Рисунок 5.2 — Сторінка кредитів коли жоден не взятий

### 5.2.2 Сторінка гаражу

Сторінка вантажівок виглядає так:

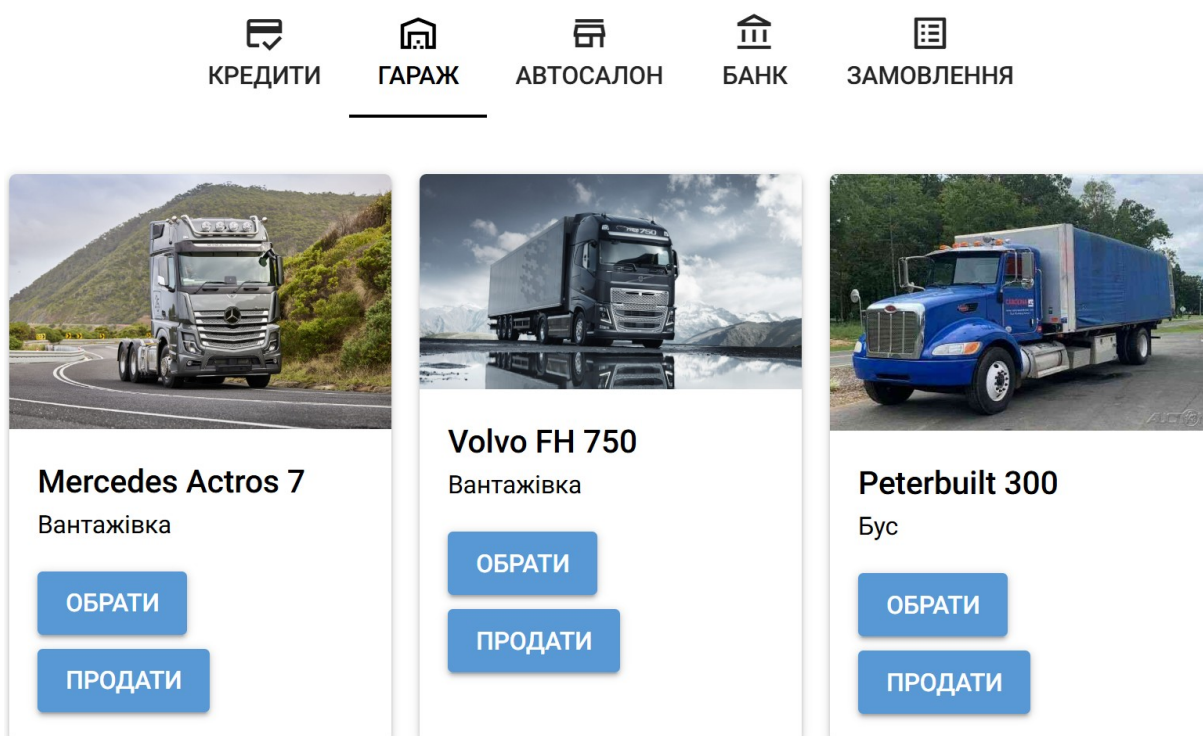


Рисунок 5.3 — Вигляд сторінки вантажівок

Вантажівку можна обрати, для цього потрібно натиснути кнопку Обрати під відповідним автомобілем.

Кожен автомобіль можна продати натисканням кнопки Продати під відповідним автомобілем.

Коли жодної вантажівки не куплено, показується наступне повідомлення:



Немає придбаних вантажівок.

Рисунок 5.4 — Вигляд сторінки вантажівок, коли жодної не куплено

5.2.3 Сторінка автосалону

Сторінка автосалону має такий вигляд:

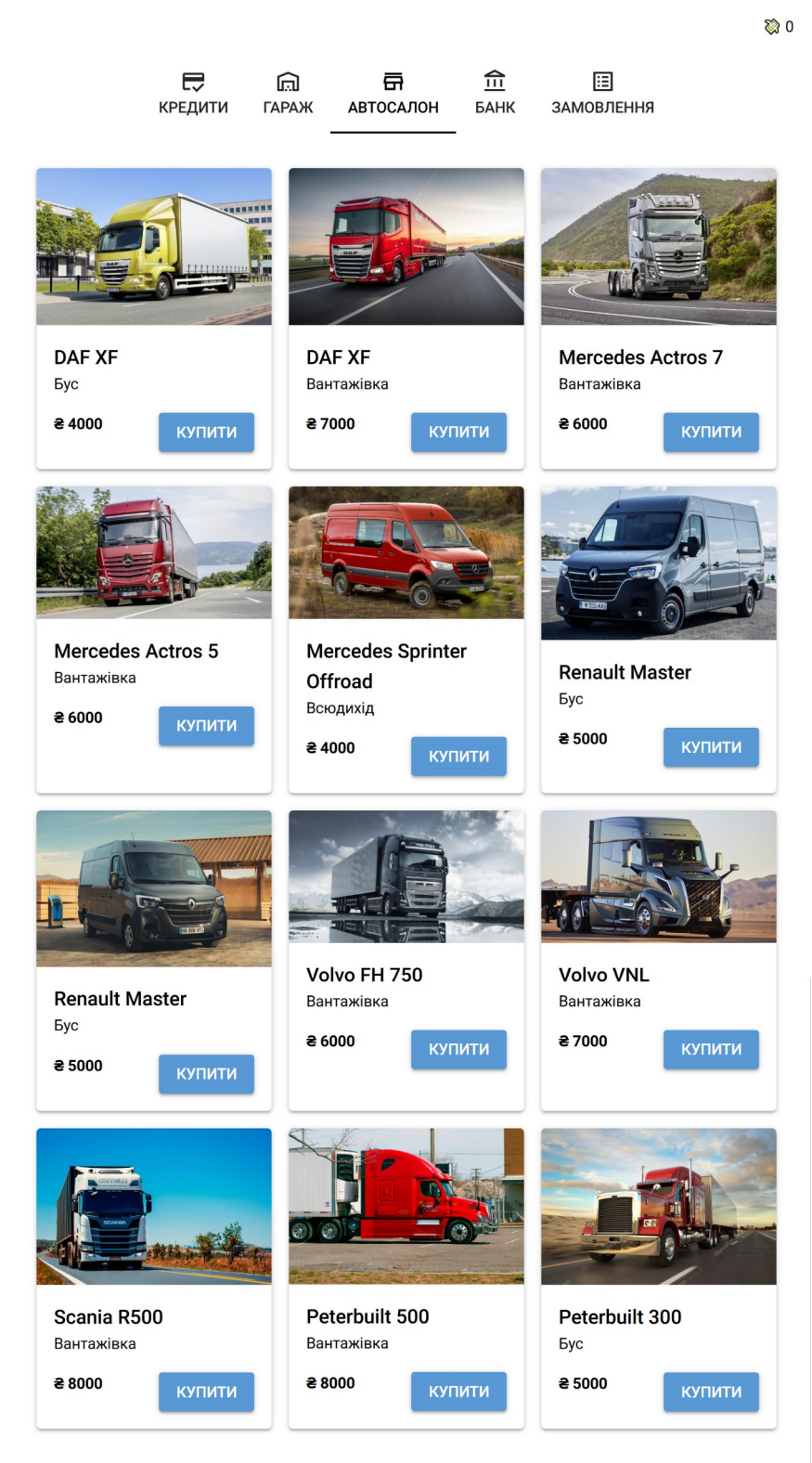


Рисунок 5.5 — Вигляд сторінки автосалону

На сторінці можна купити будь-яку вантажівку, натиснувши під нею кнопку Купити.

#### 5.2.4 Сторінка банку

Вигляд сторінки банку наведено нижче на рисунку:

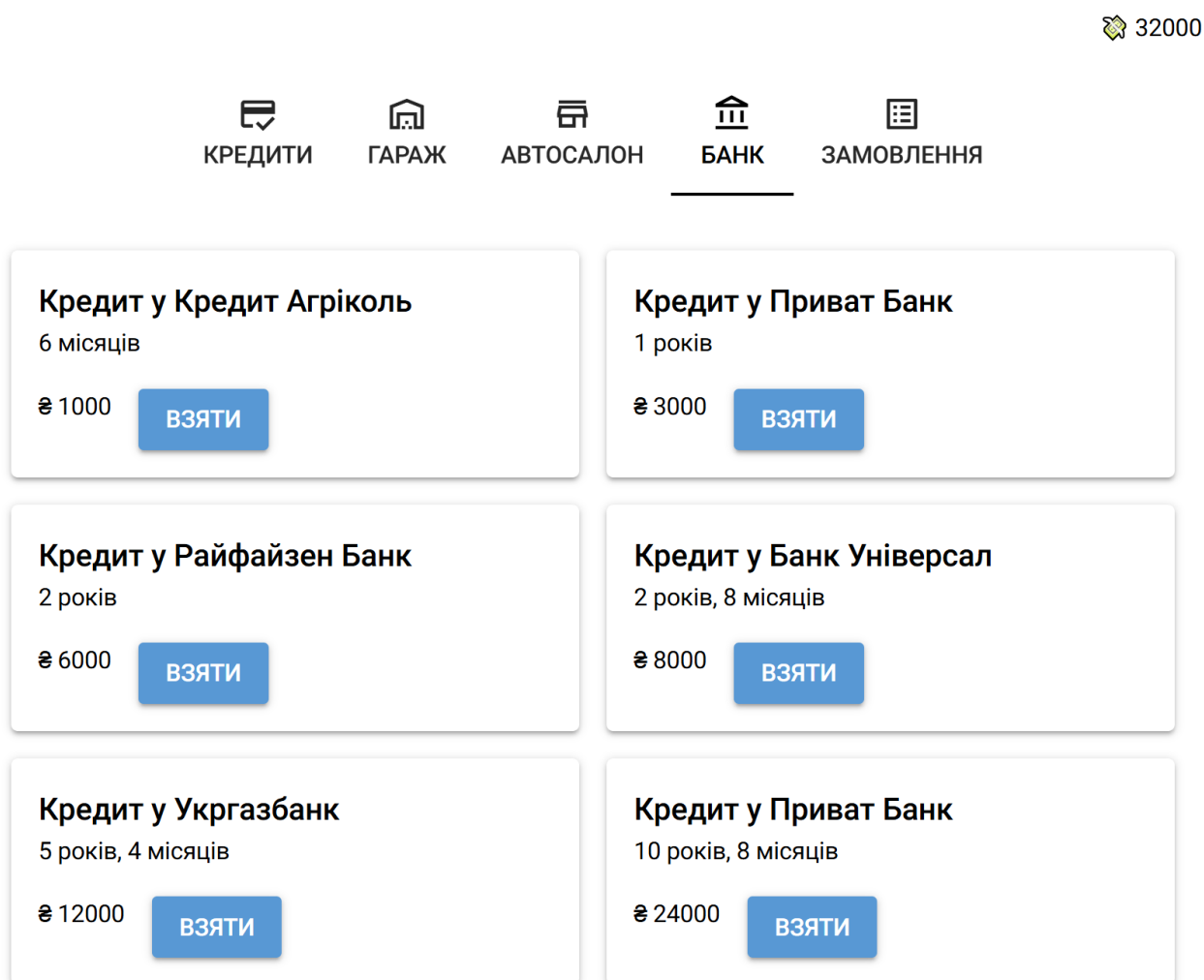


Рисунок 5.6 — Вигляд сторінки

Користувач може брати будь-яку кількість будь-якого з кредитів, натиснувши під відповідним кнопку Взяти.

### 5.2.5 Сторінка замовлень

Сторінка замовлень має наступний вигляд:

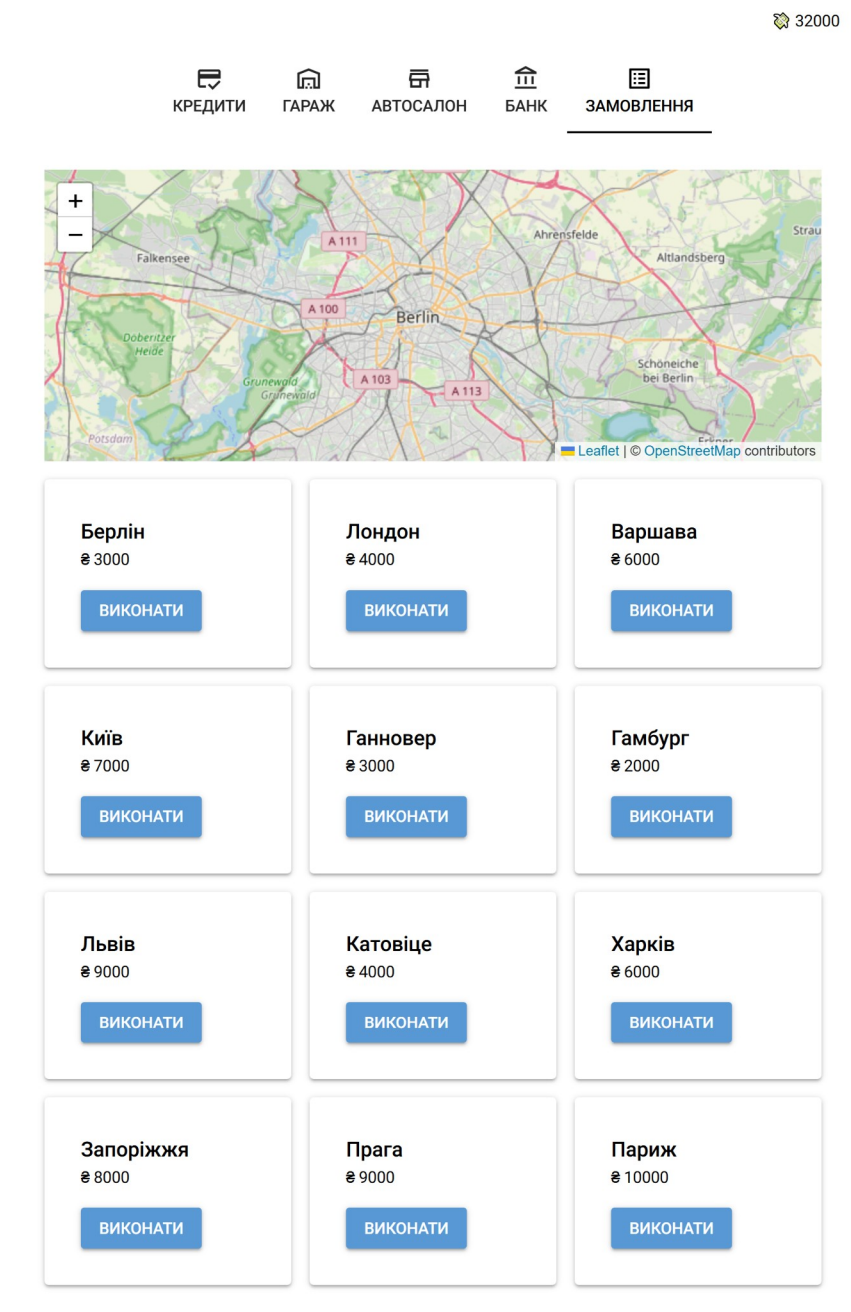


Рисунок 5.7 — Вигляд сторінки замовлень

На мапі буде показано місто, до якого здійснюється замовлення. При натисканні кнопки Виконати буде показано відповідне місто.

## ВИСНОВКИ

Благодаттю Господа Ісуса Христа виконано роботу. Було досліджено предметну область вантажних перевезень. Також було розроблено веб-застосунок, який дозволяє купувати вантажні автомобілі різних типів.

У першому розділі було оглянуто предметну область вантажних перевезень та програми аналоги. Також було розглянуто особливості програм прикладів та їх інтерфейси.

У другому розділі було розглянуто структуру програми. Було визначено функціональну та структурну схеми застосунку.

У третьому розділі було обрано засоби розробки. Їхній вибір було обґрунтовано і порівняно з іншими опціями.

У четвертому розділі було розроблено інтерфейс веб-застосунку. Було не тільки зроблено схему інтерфейсу, але й наведено вигляд готового інтерфейсу.

У п'ятому розділі було розглянуто кожен елемент інтерфейсу окремо з його функціоналом.

Надалі покращити програму можна розмістивши код на віддаленому сервері та додаванням системи гаражів та купівлю причепів.

## ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

6. Вантажні перевезення Запоріжжя: Ціна на вантажні перевезення в Запоріжжі. URL: <https://avrora-trans.com/ua/services/ukraine/gruzoperevozki-zaporozhe>. (дата звернення: 29.01.2026)

2. [AUTO.RIA - Базар авто №1: автосалони, продаж авто б/в і нових. Автопошук по 170 оголошень України. AUTO.RIA Базар авто.](#) URL: [https://auto.ria.com/uk/search/?search\\_type=1&category=6&has\\_video\\_messages=1&abroad=0&customs\\_cleared=1&page=0&limit=20](https://auto.ria.com/uk/search/?search_type=1&category=6&has_video_messages=1&abroad=0&customs_cleared=1&page=0&limit=20). (дата звернення 19.01.2026).

3. [Вантажівки Volvo Trucks.](#) URL: <https://www.volvotrucks.com.ua/uk-ua/>. (дата звернення 19.01.2026).

4. [Нові вантажівки | DAF Trucks Ukraine.](#) URL: <https://daf.ua/trucks/trucks-in-stock/new-trucks/>. (дата звернення: 19.01.2026)

5. [Продається на RST | авто Вантажні | Купити Вантажні.](#) URL: <https://rst.ua/ukr/oldcars/?k=2&task=newresults>. (дата звернення: 28.01.2026)

6. [Вантажне авто, купити вантажівку в Україні нову і б/в на OLX.](#) URL: <https://www.olx.ua/uk/transport/gruzovye-avtomobili/?currency=UAH>. (дата звернення: 28.01.2026)

**ДОДАТОК А**  
Технічне завдання

## **Вступ**

Програма має назву «Застосунок продажу вантажівок».

Програма призначена для купівлі вантажівок та їх використання для здійснення завдань.

Вона може бути використана будь-яким користувачем, якому цікаво працювати на вантажівці.

## **Підстави для розробки**

Підставою розробки є створення застосунку, який має полегшити купівлю вантажних автомобілів.

## **Призначення розробки**

Програма має дозволяти купівлю вантажівок, здійснення замовлень на них.

## **Вимоги до програми**

Основними вимогами до програми є:

- платформа — веб-браузер;
- світлий графічний інтерфейс користувача.

## ***Вимоги до функціоналу***

Програма має виконувати такі функції:

- купівля автомобілів;
- взяття кредитів;
- продаж автомобілів.

### ***Вимоги до технічних та програмних засобів***

Програма має працювати у веб-браузері. Вона має бути доступна як для мобільних, так і для десктопних платформ.

- процесор: x64 / x86 / ARM;
- оперативна пам'ять: 1 ГБ;
- накопичувач: 5 ГБ вільного місця;
- операційна система: Android, iOS, macOS, Linux, Windows.

Програмні засоби можуть змінюватися в процесі розробки, але заплановано використати такі:

- мова програмування: Python;
- фреймворк інтерфейсу: NiceGUI;
- база даних: SQLite.

### **Вимоги до програмної документації**

Програмна документація має бути реалізована у:

- пояснювальній записці;
- технічному завданні.

Код програми має бути написаний із зрозумілими назвами змінних/класів/методів і може включати коментарі.

**ДОДАТОК Б**

Текст програми

## Текст головного файлу main.py

```

from nicegui import ui
import os

from data import (
    ILoan,
    IOrder,
    TabName,
    ITruck,
    load_loans,
    load_orders,
    load_trucks,
)

class State:
    # Всі вантажівки
    trucks: list[ITruck] = list()
    # Всі кредити
    loans: list[ILoan] = list()
    # Всі замовлення
    orders: list[IOrder] = list()

    # Власні вантажівки
    owned_trucks: list[ITruck] = list()
    # Обрана вантажівка
    selected_truck: ITruck = None
    # Взяті кредити
    taken_loans: list[ILoan] = list()
    # Гроші користувача
    money: int = 0

class Const:
    CURRENT_FOLDER = os.path.dirname(os.path.abspath(__file__))

# --- ПОМІЧНИКИ ---

def convert_months_to_years(duration: int):
    MONTHS_IN_A_YEAR = 12

    years = duration // MONTHS_IN_A_YEAR
    months = duration % MONTHS_IN_A_YEAR

    return (
        # Повернути роки та місяці
        f"{years} років, {months} місяців"
        # Якщо надано обидва
        if months and years
        # Інакше == роки
        else f"{years} років"
        # Якщо їх надано
        if years
        # Інакше == місяці
        else f"{months} місяців"
    )

```

```

    )

# --- ОНОВЛЕННЯ ---

def update_money():
    money_label.set_text(f"💰 {State.money}")
    money_label.update()

def update_selected_truck():
    selected_truck_label.set_text(
        # Встановити назву обраної вантажівки, якщо вона обрана. Інакше ==
        # нуль
        f"🚚 {State.selected_truck.name}" if State.selected_truck else ""
    )
    selected_truck_label.update()

# --- ЗАМОВЛЕННЯ ---

def perform_order(order: IOrder, map):
    # Якщо вантажівки не обрано
    if State.selected_truck is None:
        # Повідомити користувача про це
        ui.notify("Немає обраної вантажівки.", close_button="Добре")
        # Завершити роботу
        return

    # Додати до грошей користувача ціну замовлення
    State.money += order.price
    # Оновити лейбл
    update_money()

    # На мапі показати місце, куди йде замовлення
    map.set_center(order.coordinates)
    map.update()

# --- ВАНТАЖІВКИ ---

def select_truck(truck: ITruck):
    # Встановити обрану вантажівку у надану
    State.selected_truck = truck
    update_selected_truck()

def buy_truck(truck: ITruck):
    # Якщо користувацьких грошей менше за вартість вантажівки
    if State.money < truck.price:
        # Повідомити про це користувача
        ui.notify("Недостатньо коштів.", close_button="Добре")
        # Завершити роботу
        return

    # Відняти від грошей користувача ціну вантажівки

```

```

State.money -= truck.price
update_money()

# До власних вантажівок додати куплену
State.owned_trucks.append(truck)
owned_trucks_view.refresh()

# Встановити нову вантажівку як обрану
select_truck(truck)
# Повідомити користувача про успішну купівлю
    ui.notify(f"Куплено {truck.category.lower()} під назвою
{truck.name}!")

def sell_truck(truck: ITruck):
    # Обнулити обрану вантажівку
    State.selected_truck = None
    update_selected_truck()

    # Прибрати вантажівку з придбаних
    State.owned_trucks.remove(truck)
    owned_trucks_view.refresh()
    # Повідомити користувача про успішний продаж
        ui.notify(f"Продано {truck.category.lower()} під назвою
{truck.name}!")

    # Додати до грошей користувача ціну вантажівки
    State.money += truck.price
    update_money()

# --- КРЕДИТИ ---

def take_loan(loan: ILoan):
    # Додати до грошей користувача суму кредиту
    State.money += loan.amount
    # До взятих кредитів додати наданий
    State.taken_loans.append(loan)

    update_money()
    # Повідомити користувача про успішне взяття кредиту
    ui.notify(f"Взято кредит на ₺ {loan.amount} від {loan.bank}")
    loans_view.refresh()

def pay_loan(loan: ILoan):
    # Якщо грошей у користувача менше за суму кредиту
    if State.money < loan.amount:
        # Повідомити про це користувача
        ui.notify("Недостатньо коштів для сплати.", close_button="Добре")
        # Завершити роботу
        return

    # Від грошей користувача відняти суму кредиту
    State.money -= loan.amount
    # Від взятих кредитів прибрати наданий
    State.taken_loans.remove(loan)

    update_money()

```

```

bank_view.refresh()
loans_view.refresh()
# Повідомити користувача про успішне сплатнення кредиту
ui.notify(f"Сплачено кредит на ₪ {loan.amount} від {loan.bank}")

# --- ПРЕДСТАВЛЕННЯ ІНТЕРФЕЙСУ ---

@ui.refreshable
def loans_view():
    # Якщо немає взятих кредитів
    if not State.taken_loans:
        # Повідомити про це користувача
        ui.label("Немає взятих кредитів.")
    with ui.grid(columns=2).classes("w-full"):
        # Для кожного взятого кредиту
        for loan in State.taken_loans:
            # Зробити картку
            with ui.card().tight():
                with ui.card_section():
                    # Назва кредиту
                    ui.label(f"Кредит у {loan.bank}").classes("font-
medium")

                    # Триаалість кредиту
                    ui.label(f"на
{convert_months_to_years(loan.duration)}")
                with ui.row().classes("mt-4"):
                    # Сума кредиту
                    ui.label(f"₪ {loan.amount}")
                    # Сплатити кредит
                    ui.button(
                        "Сплатити",
                        on_click=lambda this_loan=loan:
pay_loan(this_loan),
                    )

@ui.refreshable
def owned_trucks_view():
    if not State.owned_trucks:
        ui.label("Немає придбаних вантажівок.")
    with ui.grid(columns=3).classes("w-full"):
        # Для кожної вантажівки у придбаних
        for truck in State.owned_trucks:
            # Створити картку
            with ui.card().tight():
                # Обрахувати шлях до файлу картинки
                image_path: str = os.path.join(
                    Const.CURRENT_FOLDER, "images", f"{truck.picture}.jpg"
                )
                # Вивести картинку
                ui.image(image_path)
            with ui.card_section():
                # Назва вантажівки
                ui.label(truck.name).classes("text-lg font-medium")
                # Категорія вантажівки
                ui.label(truck.category)
            with ui.row().classes("flex flex-row gap-2 mt-4"):
                # Обрати вантажівку

```

```

        ui.button(
            "Обрати",
            on_click=lambda this_truck=truck:
select_truck(this_truck),
        )
        # Продати вантажівку
        ui.button(
            "Продати",
            on_click=lambda this_truck=truck:
sell_truck(this_truck),
        )

```

```

@ui.refreshable
def store_view():
    with ui.grid(columns=3).classes("w-full"):
        # Для кожної вантажівки у вантажівках
        for truck in State.trucks:
            # Створити картку
            with ui.card().tight():
                # Обрахувати шлях до файлу картинки
                image_path: str = os.path.join(
                    Const.CURRENT_FOLDER, "images", f"{truck.picture}.jpg"
                )
                # Вивести картинку
                ui.image(image_path)
            with ui.card_section().classes("w-full"):
                # Назва вантажівки
                ui.label(truck.name).classes("text-lg font-medium")
                # Категорія вантажівки
                ui.label(truck.category)
            with ui.row().classes("justify-between flex flex-row
w-full mt-4"):
                # Вартість вантажівки
                ui.label(f"€ {truck.price}").classes("font-bold ")
                # Купити вантажівку
                ui.button(
                    "Купити",
                    on_click=lambda this_truck=truck:
buy_truck(this_truck),
                )

```

```

@ui.refreshable
def bank_view():
    with ui.grid(columns=2).classes("w-full"):
        # Для кожного кредиту зі всіх кредитів
        for index, loan in enumerate(State.loans, start=1):
            # Створити картку
            with ui.card().tight():
                with ui.card_section():
                    # Назва кредиту
                    ui.label(f"Кредит у {loan.bank}").classes("font-medium
text-lg")
                    # Тривалість кредиту
                    ui.label(convert_months_to_years(loan.duration))
                with ui.row().classes("mt-4"):
                    # Вартість кредиту
                    ui.label(f"€ {loan.amount}")
                    # Взяти кредит

```

```

        ui.button(
            "Взяти",
            on_click=lambda this_loan=loan:
take_loan(this_loan),
        )

@ui.refreshable
def orders_view():
    # Зробити віджет мапи
    map = ui.leaflet(center=State.orders[0].coordinates, zoom=10)
    with ui.grid(columns=3).classes("w-full"):
        # Для кожного замовлення у всіх замовленнях
        for order in State.orders:
            # Зробити картку
            with ui.card():
                with ui.card_section().classes("w-full"):
                    # Пункт призначення замовлення
                    ui.label(order.location).classes("text-lg font-
medium")

                    # Вартість замовлення
                    ui.label(f"₾ {order.price}").classes("mb-4")
                    # Виконати замовлення
                    ui.button(
                        "Виконати",
                        on_click=lambda this_order=order: perform_order(
                            this_order, map
                        ),
                    )

State.trucks = load_trucks()
State.loans = load_loans()
State.orders = load_orders()

with ui.row().classes("flex flex-row gap-4 self-end"):
    selected_truck_label = ui.label(
        f"
```

## Текст додаткового модулю data.py

```

from dataclasses import dataclass
from peewee import *

class TabName:
    LOANS = "Кредити"
    OWNED = "Гараж"
    STORE = "Автосалон"
    BANK = "Банк"
    ORDERS = "Замовлення"

class TruckCategory:
    LOCAL = "Бус"
    LORRY = "Вантажівка"
    OFFROAD = "Всюдихід"

class BankName:
    PRIVAT = "Приват Банк"
    RAIFFEISEN = "Райфайзен Банк"
    CREDIT_AGRICOLE = "Кредит Агріколь"
    UNIVERSAL = "Банк Універсал"
    UKR_GAS_BANK = "Укргазбанк"

@dataclass
class ITruck:
    name: str
    category: TruckCategory
    price: int
    picture: str

@dataclass
class ILoan:
    amount: int
    duration: int
    bank: BankName

@dataclass
class IOrder:
    price: int
    location: str
    coordinates: tuple[float, float]

db_name = "database.db"
db = SqliteDatabase(db_name)

class BaseModel(Model):
    class Meta:
        database = db

```

```

class Truck(BaseModel):
    name = CharField()
    category = CharField()
    price = IntegerField()
    picture = CharField()

class Loan(BaseModel):
    amount = IntegerField()
    duration = IntegerField()
    bank = CharField()

class Order(BaseModel):
    price = IntegerField()
    location = CharField()
    coordinates = CharField()

db.connect()

def load_orders() -> list[IOrder]:
    orders: list[IOrder] = list()

    all_orders_query = Order.select()
    for row in all_orders_query:
        order = IOrder(
            price=row.price,
            location=row.location,
            coordinates=row.coordinates.split(","),
        )
        orders.append(order)

    return orders

def load_loans() -> list[ILoan]:
    loans: list[ILoan] = list()

    all_loans_query = Loan.select()
    for row in all_loans_query:
        loan = ILoan(amount=row.amount, duration=row.duration, bank=row.bank)
        loans.append(loan)

    return loans

def load_trucks() -> list[ITruck]:
    trucks: list[ITruck] = list()

    all_trucks_query = Truck.select()
    for row in all_trucks_query:
        truck = ITruck(
            name=row.name, category=row.category, price=row.price,
            picture=row.picture
        )
        trucks.append(truck)

    return trucks

```

**ДОДАТОК В**  
Слайди презентації

# Продаж Вантажівок

Дипломна робота

доцент, Степаненко О.О.

студент КНТ-122, Онищенко О.А.

Рисунок 1 — Слайд 1

**Зміст**

**01**

Презентація складається з таких частин:

1. Порівняння фреймворків
2. Порівняння середовищ розробки
3. Огляд прикладів
4. Приклади використання

Рисунок 2 — Слайд 2

Порівняння фреймворків

02

| Критерій         | NiceGUI     | Streamlit   | Tkinter |
|------------------|-------------|-------------|---------|
| Платформа        | Веб-браузер | Веб-браузер | Десктоп |
| Персоналізація   | Висока      | Низька      | Середня |
| Швидкість роботи | Висока      | Середня     | Середня |

Рисунок 3 — Слайд 3

Порівняння середовищ розробки

03

| Критерій                | Visual Studio Code | Notepad | neovim  |
|-------------------------|--------------------|---------|---------|
| Зрозумілість інтерфейсу | Середня            | Висока  | Низька  |
| Легкість освоєння       | Середня            | Висока  | Низька  |
| Зручність               | Висока             | Середня | Середня |
| Додаткові плагіни       | Висока             | Низька  | Середня |
| Підтримка спільноти     | Висока             | Низька  | Середня |

Рисунок 4 — Слайд 4

Застосунок приклад: AutoRIA

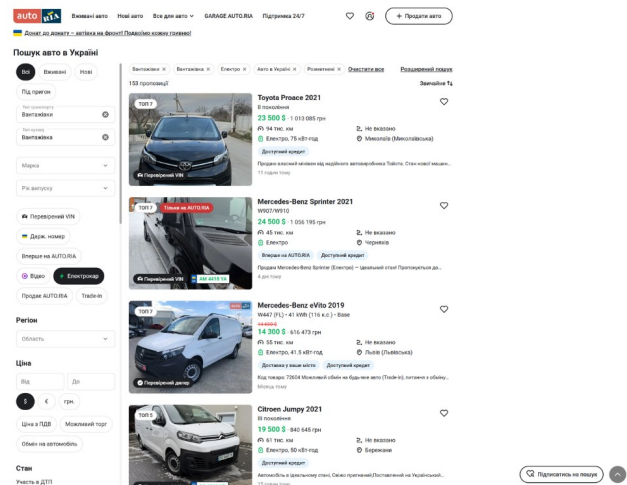


Рисунок 5 — Слайд 5

Застосунок приклад: Volvo

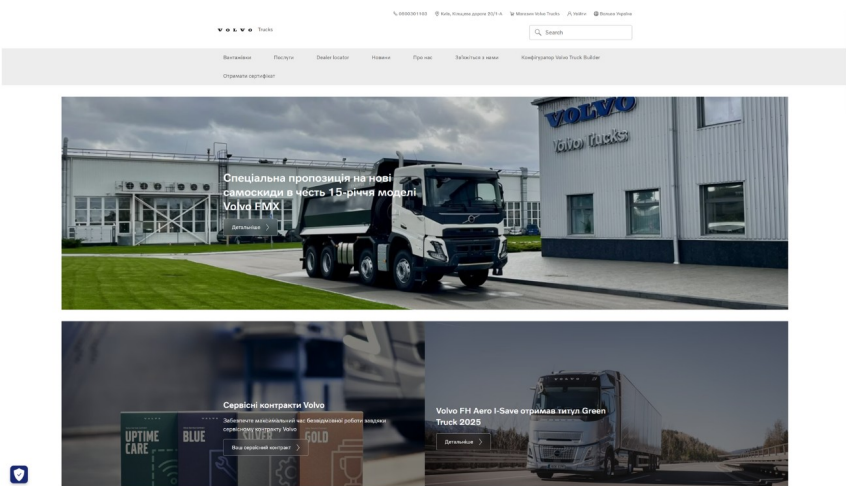


Рисунок 6 — Слайд 6

## Застосунок приклад: DAF

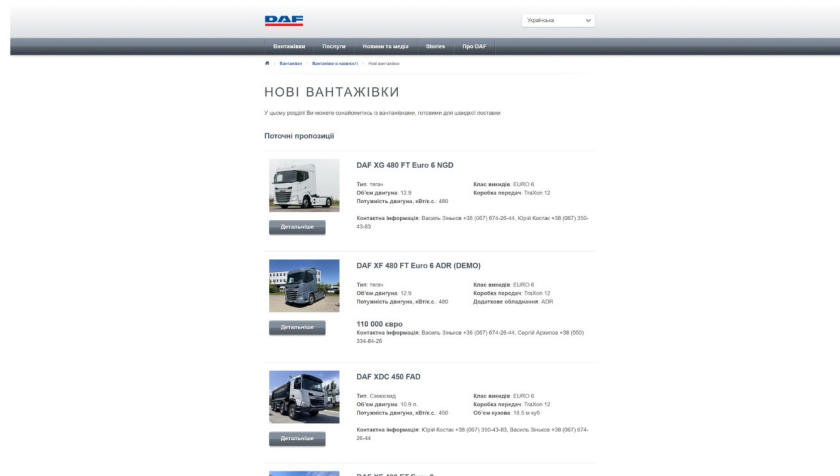


Рисунок 7 — Слайд 7

## Застосунок приклад: RST

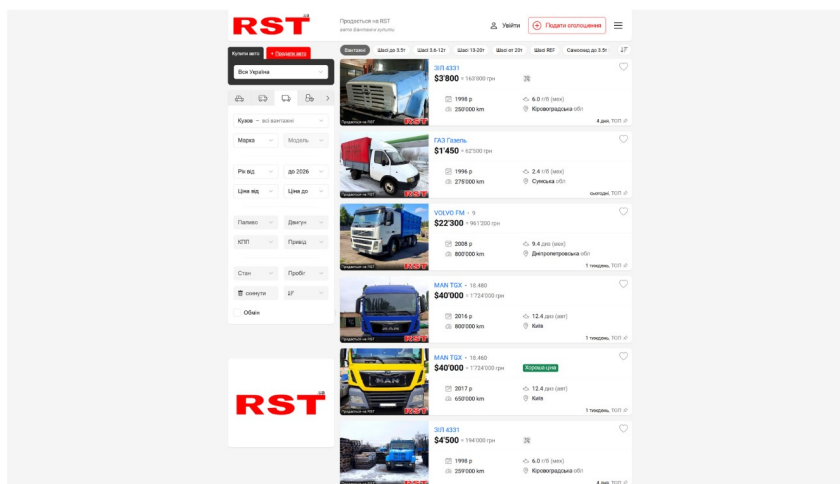


Рисунок 8 — Слайд 8

Застосунок приклад: OLX

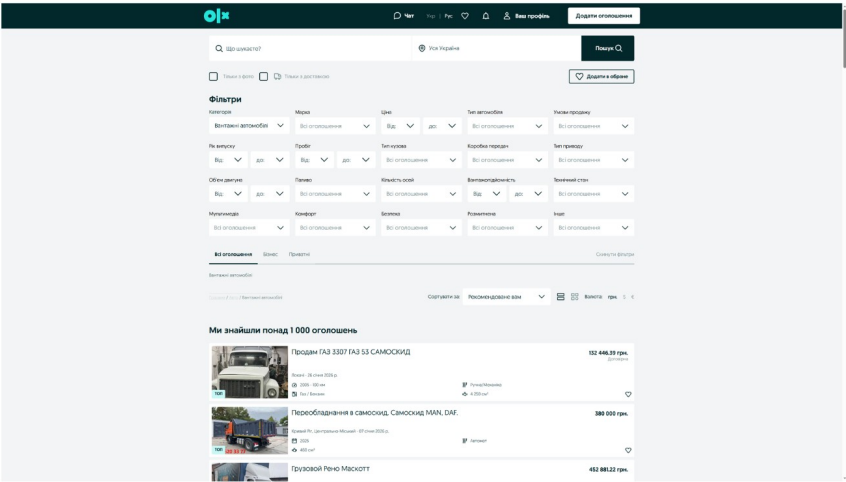


Рисунок 9 — Слайд 9

Вигляд застосунку: кредити

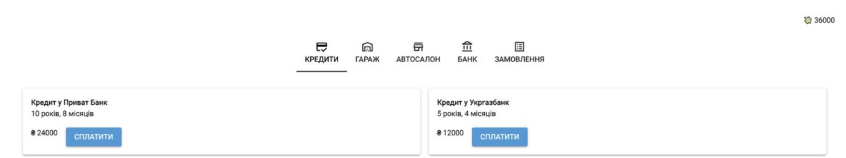


Рисунок 10 — Слайд 10

Вигляд застосунку: гараж

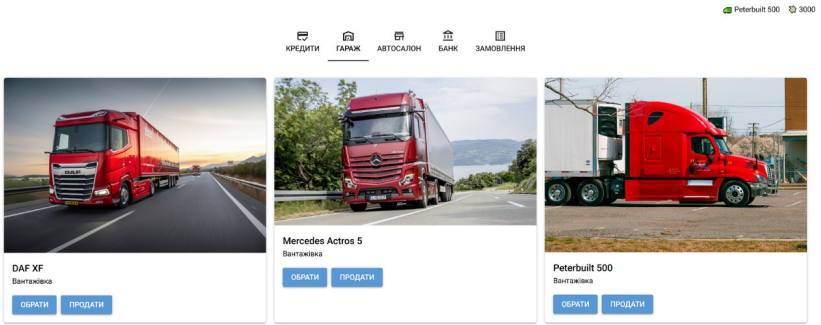


Рисунок 11 — Слайд 11

Вигляд застосунку: автосалон

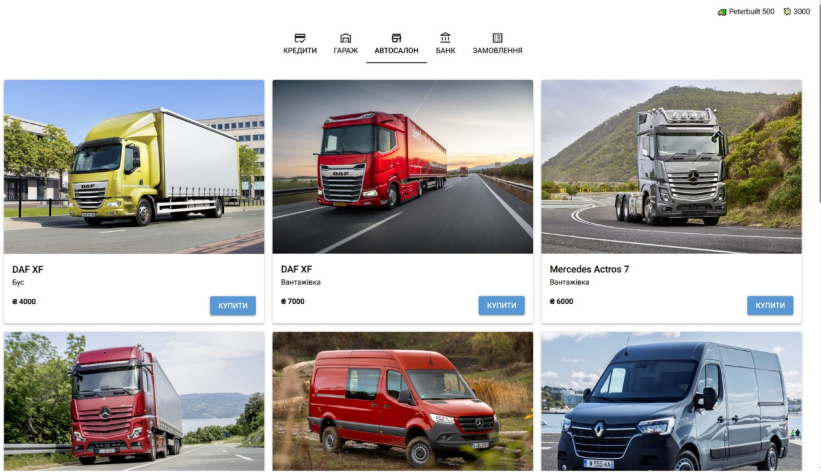


Рисунок 12 — Слайд 12

Вигляд застосунку: банк

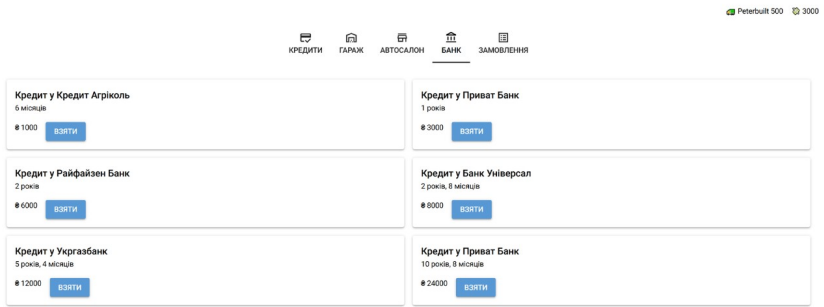


Рисунок 13 — Слайд 13

Вигляд застосунку: замовлення

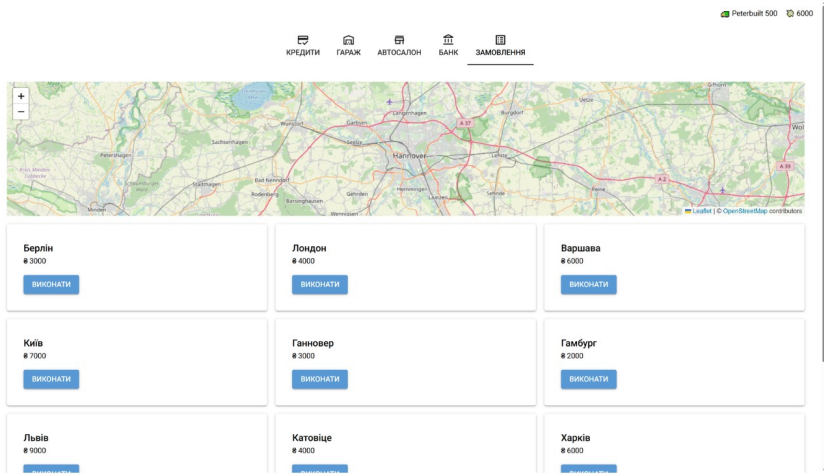


Рисунок 14 — Слайд 14

## Висновки

14

**Так бо Бог полюбив світ, що дав  
Сина Свого Однородженого, щоб  
кожен, хто вірує в Нього, не  
згинув, але мав життя вічне.**

*Івана 3:16*

Рисунок 15 — Слайд 15